

Cahier des Charges Fonctionnel — NautiLog

Projet : NautiLog — Gestion de flotte nautique et carte interactive de disponibilité **Titre visé :** Concepteur Développeur d'Applications (CDA) **Auteur :** Mell CANAC

Version du document : 1.0

Sommaire

1. Introduction et contexte
 2. Objectifs du projet
 3. Public cible et rôles
 4. Spécifications fonctionnelles
 5. Architecture technique
 6. Sécurité
 7. Risques et contraintes
 8. Exigences non fonctionnelles
 9. Conclusion et perspectives
 10. Annexe — Glossaire
-

1. Introduction et contexte

1.1 Contexte du projet

NautiLog est une application de gestion de flotte nautique développée dans le cadre d'un titre professionnel Concepteur Développeur d'Applications (CDA), en solo, sur une durée de 10 semaines. Le projet répond à un besoin concret dans le secteur de la location et de la gestion de bateaux de plaisance : centraliser, sur une plateforme unique, la gestion d'une flotte de bateaux, leur localisation dans des ports, leur statut de disponibilité, et les réservations effectuées par des locataires.

1.2 Problématique adressée

Les gestionnaires de flotte nautique (loueurs, capitaineries, propriétaires multi-bateaux) jonglent souvent avec des outils disparates (tableurs, échanges par email/téléphone) pour suivre :

- la disponibilité réelle de chaque bateau (disponible, loué, en réparation) ;
- l'emplacement précis d'un bateau dans un port (ponton, quai) ;
- l'historique des réparations et de la maintenance ;
- les réservations et leur statut.

NautiLog centralise ces informations dans une application web sécurisée, avec des rôles distincts selon le profil de l'utilisateur (administrateur, propriétaire, locataire).

1.3 Valeur ajoutée

Axe	Apport de NautiLog
Centralisation	Une seule source de vérité pour la flotte, les ports et les réservations
Visibilité géographique	Carte interactive (Leaflet) des ports et des emplacements de bateaux
Sécurité des données	Chiffrement des données sensibles (numéro de permis), JWT, RBAC
Conformité RGPD	Droit à l'oubli effectif (suppression de compte en cascade)
Qualité et fiabilité	Pipeline CI/CD avec tests automatisés et scan de vulnérabilités

1.4 Choix techniques (résumé)

Le projet repose sur une architecture API-first : un backend Symfony 8.1 exposant une API REST, consommée par un frontend React/Vite découplé, le tout conteneurisé avec Docker Compose. Ce choix répond directement aux exigences de séparation en couches du référentiel CDA (CCP2) et permet une évolution indépendante du frontend et du backend. Le détail est développé en section 5.

2. Objectifs du projet

2.1 Cadre méthodologique des objectifs (OPAS)

Les objectifs du projet NautiLog ont été formalisés selon la méthodologie **OPAS** — *Objectifs Paramétrés, Process Automatisé, Architecture Élastique, Séquencement Compartimenté*. Cette démarche a été spécifiquement retenue pour répondre aux exigences d'un cycle de développement individuel contraint à 10 semaines.

Plutôt que des intentions de conception vagues ou difficilement mesurables, OPAS impose des critères d'évaluation objectifs, auditables et quantifiables à chaque étape du cycle de vie du logiciel — garantissant un suivi continu de l'avancement sans sacrifier la rigueur d'ingénierie.

Critère OPAS	Spécification	Application concrète à NautiLog
O — Objectifs Paramétrés	Spécifications explicites et périmètre délimité	Découpage fonctionnel fin formalisé par un système de tickets (BE-XX/FE-XX/DEVOPS-XX) ; chaque tâche fait l'objet d'un périmètre explicite validé avant implémentation
P — Process Automatisé	Assurance qualité continue et traçabilité intégrale	Pipeline CI/CD validant systématiquement lint et tests automatisés à chaque push, couplé à un historique Git structuré (Conventional Commits)
A — Architecture Élastique	Conception modulaire et capacité d'extension	Pattern en couches (DTO / Controller / Service, cf. section 5.3) assurant un découplage fort, permettant d'ajouter une fonctionnalité métier (ex. météo marine, chiffrement) sans réécriture ni régression
S — Séquencement Compartimenté	Gestion des risques et maîtrise de l'échéancier	Développement organisé en phases délimitées sur 10 semaines, priorisation stricte des briques logicielles

Critère OPAS	Spécification	Application concrète à NautiLog
		pour sécuriser la livraison du MVP (section 2.3) et le respect des jalons de rendu

Justification et bénéfices de la démarche. L'alignement sur la grille OPAS transforme les contraintes du projet — développement solo, délais restreints — en leviers d'efficacité : la paramétrisation des tickets prévient toute dérive de périmètre ; l'automatisation des contrôles en intégration continue garantit un socle de code stable, prêt pour la démonstration ; l'élasticité de l'architecture assure enfin que NautiLog reste une plateforme évolutive au-delà du cadre initial des 10 semaines (cf. perspectives, section 9.2).

2.2 Objectifs pédagogiques (référentiel CDA)

Le projet vise à démontrer la maîtrise des trois blocs de compétences du titre CDA :

Bloc	Intitulé	Illustré par
CCP1	Développer une application sécurisée	JWT, RBAC, chiffrement AES-256-CBC, headers de sécurité, DTO/Validators
CCP2	Concevoir et développer en couches	Architecture Frontend/API/BDD, Voters Symfony, base de données relationnelle
CCP3	Préparer le déploiement d'une application	Tests automatisés, pipeline CI/CD, scan de sécurité Trivy, conteneurisation Docker

2.3 Objectifs fonctionnels (MVP)

1. Authentification sécurisée et gestion de rôles (administrateur, propriétaire, locataire).
2. Gestion complète d'une flotte de bateaux (création, modification, suivi de statut, historique de réparations).
3. Gestion des ports et des emplacements d'amarrage, avec visualisation cartographique.
4. Système de réservation avec détection des conflits de dates.
5. Conformité RGPD (droit à l'oubli) et export de documents (carnet de navigation PDF).

3. Public cible et rôles

3.1 Segmentation des utilisateurs

NautiLog s'adresse à trois profils d'utilisateurs distincts, gérés par un système de contrôle d'accès basé sur les rôles (RBAC — Role-Based Access Control).

Rôle applicatif	Profil utilisateur	Besoins principaux
ROLE_ADMIN	Administrateur / gestionnaire de flotte	Vue d'ensemble de la flotte, gestion des bateaux/ports/utilisateurs, arbitrage des réservations et des litiges

Rôle applicatif	Profil utilisateur	Besoins principaux
ROLE_OWNER	Propriétaire de bateau(x)	Gestion de son propre parc de bateaux, suivi des réparations, consultation des réservations le concernant
ROLE_RENTER	Locataire	Recherche et réservation d'un bateau disponible, suivi de ses réservations

Un rôle technique complémentaire, **ROLE_USER**, sert de socle commun à tout utilisateur authentifié (accès aux routes nécessitant une simple authentification, indépendamment du rôle métier).

3.2 Matrice des droits d'accès (extrait significatif)

Action	Admin	Propriétaire	Locataire
Consulter la liste des bateaux et ports (public)	✓	✓	✓
Créer / modifier / supprimer un bateau	✓	✓ (le sien)	✗
Changer le statut d'un bateau en "En réparation"	✓	✗	✗
Gérer les emplacements (Berth) d'un port	✓	✗	✗
Réserver un bateau disponible	—	—	✓
Consulter le dashboard admin (KPIs, gestion utilisateurs)	✓	✗	✗
Supprimer son propre compte (RGPD)	✓	✓	✓

Le contrôle des droits est appliqué à deux niveaux : globalement via la configuration **access_control** de Symfony Security, et finement au cas par cas via un Voter Symfony dédié (**BoatVoter**, attributs **BOAT_EDIT/BOAT_DELETE**) qui autorise une action si l'utilisateur est administrateur **ou** propriétaire du bateau concerné.

4. Spécifications fonctionnelles

4.1 Authentification et gestion de compte

- Inscription et connexion par email/mot de passe, authentification par jeton JWT (stateless).
- Page de profil utilisateur : consultation et édition des informations personnelles.
- Suppression de compte conforme RGPD (droit à l'oubli), détaillée en section 6.4.

4.2 Gestion de la flotte

- CRUD complet des bateaux : nom, type, statut (disponible / loué / en réparation), photo, port d'attache.
- Historique des réparations par bateau (entité **Repair**), avec restriction : seul un administrateur peut positionner un bateau en statut "En réparation".
- Export d'un carnet de navigation au format PDF pour un bateau donné.

4.3 Gestion des ports et des emplacements

- CRUD des ports (nom, ville, coordonnées géographiques, capacité).
- Gestion des emplacements d'amarrage (**Berth**) au sein d'un port, avec attribution d'un bateau.
- Système de demande d'emplacement (**BerthRequest**) : un propriétaire formule une demande, un administrateur l'approuve ou la refuse.
- Carte interactive (Leaflet/OpenStreetMap) affichant les ports et, à l'intérieur de chaque port, les emplacements avec une icône différenciée par type de bateau ; la visibilité des emplacements occupés est filtrée par rôle (un administrateur voit tous les bateaux amarrés, un propriétaire ne voit que les siens).
- Affichage de la météo marine actuelle (température, vent, état de la mer estimé) par port, via l'API publique Open-Meteo, proxifiée par le backend.

4.4 Réservations

- Un locataire peut réserver un bateau disponible sur une plage de dates.
- Détection automatique des chevauchements de dates pour un même bateau.
- Vue de gestion des réservations côté propriétaire et administrateur (confirmation, annulation).

4.5 Signalements et notifications

- Un utilisateur peut signaler un problème lié à une réservation ; l'administrateur y répond, ce qui notifie l'auteur.
- Système de notifications internes (ex. réponse à un signalement, suppression de compte d'un propriétaire impactant un locataire actif) avec compteur de notifications non lues dans la barre de navigation.

4.6 Portail d'administration

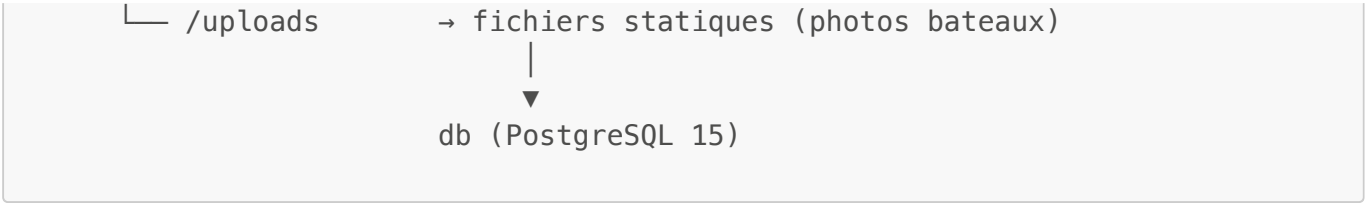
- Dashboard **Overview** : indicateurs clés (nombre de bateaux par statut, nombre de ports, nombre d'utilisateurs par rôle, signalements en attente) et carte interactive globale de la flotte.
- Gestion des utilisateurs : liste, consultation, modification du rôle.
- Gestion de la flotte et des ports avec opérations groupées (suppression multiple, tri par dernière mise à jour).

5. Architecture technique

5.1 Vue d'ensemble

NautiLog suit une architecture **API-first découplée** : un frontend SPA (Single Page Application) consomme une API REST backend, les deux étant indépendamment conteneurisés et communiquant via HTTP/JSON derrière un reverse-proxy Nginx.





5.2 Stack technique détaillée

Couche	Technologie	Version
Langage backend	PHP	≥ 8.4
Framework backend	Symfony	8.1
ORM	Doctrine ORM	3.x
Authentification	LexikJWTAuthenticationBundle	—
Génération PDF	Dompdf	^3.1
Base de données	PostgreSQL	15 (alpine)
Langage frontend	JavaScript (React)	React 18.3
Outil de build frontend	Vite	^5.3
Client HTTP	Axios	^1.7
Cartographie	React Leaflet / Leaflet	^4.2 / ^1.9
Routage frontend	React Router DOM	^6.23
Styles	Tailwind CSS	^3.4
Runtime frontend (build/dev)	Node.js	20
Conteneurisation	Docker / Docker Compose	—
Reverse-proxy	Nginx	alpine
Capture d'emails (dev)	Mailpit	latest

5.3 Architecture en couches (backend)

Le backend applique une séparation stricte des responsabilités, conforme aux exigences CCP2 :

Couche	Rôle	Exemple
Controller	Réception de la requête HTTP, orchestration	BateauController, PortController
DTO (Data Transfer Object)	Validation et transport des données entrantes	BateauDto, PortDto
Validator	Règles de validation métier (contraintes Symfony)	Annotations #[Assert\...] sur les DTO

Couche	Rôle	Exemple
Service	Logique métier réutilisable, hors HTTP	<code>EncryptionService</code> , <code>SuppressionCompteService</code> , <code>WeatherService</code>
Entity / Repository	Persistance des données	Entités Doctrine, Repositories
Voter	Autorisation fine, à la ressource	<code>BoatVoter</code>

5.4 Modèle de données (aperçu)

Neuf entités Doctrine structurent le domaine métier : `User`, `Boat`, `Port`, `Berth`, `BerthRequest`, `Repair`, `Reservation`, `Notification`, `Signalement`. Le détail du modèle (MCD/MLD/MPD, dictionnaire de données) fait l'objet d'un document dédié (DOC-04 — Modélisation de la Base de Données).

5.5 Environnement Docker

L'environnement de développement repose sur 5 services orchestrés par Docker Compose :

Service	Image / build	Rôle
<code>db</code>	<code>postgres:15-alpine</code>	Base de données relationnelle
<code>api</code>	Build custom (PHP-FPM 8.4)	API Symfony
<code>client</code>	Build custom (Node 20)	Serveur de développement Vite
<code>nginx</code>	<code>nginx:alpine</code>	Reverse-proxy, point d'entrée unique (port 80)
<code>mailhog</code>	<code>axllent/mailpit:latest</code>	Capture des emails sortants en développement

5.6 Intégration continue / Déploiement continu

Un pipeline GitHub Actions (`ci.yml`) s'exécute sur chaque push/pull request vers `develop`, `preprod` et `main`, structuré en 4 jobs :

Job	Contenu
<code>lint</code>	<code>composer validate</code> , vérification syntaxique PHP, PHP-CS-Fixer (dry-run), ESLint frontend
<code>tests-phpunit</code>	Base PostgreSQL éphémère, migrations, exécution de la suite PHPUnit
<code>trivy-scan</code>	Scan de vulnérabilités du filesystem (sévérités HIGH/CRITICAL), non bloquant
<code>build-docker</code>	Build et publication des images <code>backend</code> / <code>frontend</code> sur GitHub Container Registry (dépend des deux premiers jobs)

6. Sécurité

6.1 Authentification

L'authentification repose sur des jetons JWT signés par paire de clés RSA (LexikJWTAuthenticationBundle), dans un firewall API entièrement **stateless** — aucune session serveur, cohérent avec l'architecture découplée frontend/backend.

6.2 Contrôle d'accès

Le contrôle d'accès est appliqué à deux granularités complémentaires :

- **Grossier**, au niveau des routes, via la configuration `access_control` de Symfony Security (ex. `/api/admin` réservé à `ROLE_ADMIN`, endpoints publics en lecture seule sur bateaux/ports).
- **Fin**, au niveau de la ressource, via le Voter `BoatVoter`, qui vérifie que l'utilisateur courant est bien propriétaire du bateau qu'il tente de modifier ou supprimer (sauf s'il est administrateur).

6.3 Protection des données sensibles

Le numéro de permis bateau (`User::$licenseNumber`) est chiffré en base de données via un mécanisme dédié :

Élément	Détail
Algorithme	AES-256-CBC
Vecteur d'initialisation	Aléatoire, 16 octets, généré à chaque chiffrement
Déclenchement	Attribut PHP <code>#[Encrypted]</code> posé sur la propriété, traité automatiquement par un <code>EventListener</code> Doctrine (<code>prePersist</code> , <code>preUpdate</code> , <code>postLoad</code>)
Clé de chiffrement	Externalisée en variable d'environnement (jamais committée)

Cette approche évite toute logique de chiffrement dispersée dans les contrôleurs : il suffit d'annoter une nouvelle propriété sensible pour qu'elle soit automatiquement protégée.

6.4 Conformité RGPD

Le droit à l'oubli est implémenté via `DELETE /api/me`, avec une suppression **immédiate** (sans validation humaine intermédiaire, le RGPD ne l'exigeant pas) et **en cascade complète** :

1. Pour chaque bateau possédé : notification des locataires ayant une réservation active (avec suggestion de bateaux alternatifs dans le même port), suppression des réparations, détachement des emplacements, suppression du bateau.
2. Suppression des réservations où l'utilisateur est locataire.
3. Notification des administrateurs du départ du compte.
4. Suppression des notifications adressées à l'utilisateur, puis suppression du compte.

6.5 Sécurité applicative et infrastructure

Mesure	Implémentation
Validation des entrées	DTO + Validators Symfony sur chaque endpoint d'écriture
Headers HTTP de sécurité	<code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , <code>X-XSS-Protection</code> , <code>Referrer-Policy</code> , <code>Content-Security-Policy</code> , <code>Permissions-Policy</code> (configurés au niveau Nginx)
Masquage de la stack	<code>server_tokens off</code> (Nginx)
Scan de vulnérabilités	Trivy (scan filesystem) intégré au pipeline CI
Tests de sécurité applicatifs	Suite de tests dédiée (<code>OwaspTest</code>) couvrant des scénarios OWASP de base

6.6 Limite assumée

Le rate limiting (limitation du nombre de requêtes) n'est pas implémenté à ce stade, ni au niveau Nginx ni au niveau applicatif Symfony. Ce choix est documenté comme une limite assumée du périmètre MVP plutôt qu'un oubli, à mentionner explicitement en soutenance.

7. Risques et contraintes

7.1 Matrice des risques

Risque	Probabilité	Impact	Mitigation
Délai de 10 semaines trop court pour le périmètre complet	Moyenne	Élevé	MVP strictement défini, fonctionnalités secondaires écartées ou reportées (ex. déploiement VPS)
Développement solo — pas de relecture de code par des pairs	Élevée	Moyen	Tests automatisés systématiques, CI bloquante sur le lint et les tests
Dette de configuration non détectée (ex. lint non exécuté sur du code existant)	Moyenne	Faible	Documentée en section Obstacles techniques rencontrés ; correctifs tracés par commit dédié
Absence de rate limiting	Faible (contexte démo/jury)	Moyen	Limite assumée et documentée, non bloquante pour la soutenance
Perte de disponibilité de l'API météo externe (Open-Meteo)	Faible	Faible	Le backend échoue silencieusement (retour <code>null</code> journalisé), sans impacter le reste de l'application

7.2 Contraintes du projet

Type de contrainte	Détail
Temporelle	Calendrier de rendus fixe (dépôt numérique, dépôt papier, soutenance)
Organisationnelle	Développement en solo, sans équipe ni relecture de pair
Technique	Environnement local Docker uniquement — pas d'hébergement de production réel (choix de scope assumé)
Pédagogique	Le projet doit démontrer explicitement les compétences des 3 blocs CCP du référentiel CDA

7.3 Choix de scope assumé — non-déploiement en production

Le déploiement effectif sur un VPS avec nom de domaine et certificat HTTPS a été envisagé puis abandonné, faute d'infrastructure disponible dans le périmètre du projet (achat/hébergement hors budget). La compétence de déploiement/CI est démontrée par le pipeline CI/CD fonctionnel (lint, tests, scan de sécurité, build et publication d'images Docker), jugé suffisant pour la validation CCP3 dans le cadre d'un titre CDA solo.

8. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Requêtes API répondant en conditions normales sans pagination lourde (volumétrie de démonstration, pas de charge en production réelle)
Accessibilité	Éléments RGAA de base : lien d'évitement ("aller au contenu principal"), attributs ARIA sur les composants interactifs (menu mobile, modales), navigation clavier des modales
Qualité de code	Lint bloquant en CI (PHP-CS-Fixer, ESLint), convention de nommage cohérente (voir Annexe — Glossaire)
Portabilité	Environnement entièrement conteneurisé, reproductible via <code>docker compose up</code>
Sécurité des secrets	Clés JWT et clé de chiffrement externalisées en variables d'environnement, jamais commitées
Traçabilité	Historique Git complet en Conventional Commits, journal des décisions techniques (<code>docs/decisions.md</code>)

9. Conclusion et perspectives

9.1 Bilan

NautiLog démontre, sur un périmètre fonctionnel complet (authentification, gestion de flotte, cartographie, réservations, conformité RGPD), la mise en œuvre des trois blocs de compétences du titre CDA : une application sécurisée (CCP1), architecturée en couches avec séparation claire des responsabilités (CCP2), et intégrée dans un pipeline de tests et de déploiement continu (CCP3).

9.2 Perspectives d'évolution (post-MVP)

Piste d'évolution	Intérêt
Déploiement en production réel (VPS, HTTPS)	Compléter la démonstration CCP3 en conditions réelles
Mise en place d'un rate limiting	Renforcer la robustesse face aux abus
Prévisions météo multi-jours	Enrichir la fonctionnalité météo marine actuelle
Notifications en temps réel (WebSocket/Mercure)	Remplacer le rafraîchissement manuel par du push
Application mobile	Étendre l'accès à la plateforme en mobilité

Annexe — Glossaire

Convention de nommage du projet (validée dès le début du développement) : les **entités Doctrine et repositories** sont nommés en **anglais**, tandis que les **contrôleurs, DTO et routes API** sont nommés en **français**. Ce choix répond à un double objectif pédagogique : démontrer la maîtrise du vocabulaire technique anglophone standard dans l'écosystème Symfony/Doctrine, tout en conservant une API et une interface intégralement en français pour l'utilisateur final et le jury.

Terme (entité, anglais)	Signification	Équivalent français côté API/UI
User	Utilisateur de la plateforme	Géré via <code>AuthController</code> / <code>ProfilController</code>
Boat	Bateau	<code>BateauController</code> , <code>BateauDto</code> , route <code>/api/bateaux</code>
Port	Port de plaisance	<code>PortController</code> , <code>PortDto</code> , route <code>/api/ports</code>
Berth	Poste d'amarrage / emplacement d'accostage (terme nautique standard, distinct de <i>mooring</i> et <i>slip</i>)	<code>BerthController</code> , <code>BerthDto</code> , route <code>/api/ports/{id}/emplacements</code>
BerthRequest	Demande d'attribution d'un emplacement	Route <code>/api/demandes-emplacements</code>
Repair	Réparation / maintenance d'un bateau	Route <code>/api/bateaux/{id}/reparations</code>
Reservation	Réservation d'un bateau par un locataire	Route <code>/api/reservations</code>
Notification	Notification interne à un utilisateur	Route <code>/api/notifications</code>

Terme (entité, anglais)	Signification	Équivalent français côté API/UI
Signalement	Signalement d'un litige lié à une réservation	Route <code>/api/signalements</code> (nommage déjà français, entité incluse)

RBAC — *Role-Based Access Control* : modèle de contrôle d'accès où les permissions sont attribuées à des rôles plutôt qu'individuellement à chaque utilisateur.

JWT — *JSON Web Token* : jeton d'authentification signé, transmis par le client à chaque requête, permettant une authentification sans état côté serveur (stateless).

DTO — *Data Transfer Object* : objet dédié au transport et à la validation des données échangées avec l'API, découplé de l'entité de persistance.

Voter — Mécanisme Symfony d'autorisation fine, permettant de définir une logique d'accès personnalisée à une ressource précise (au-delà du simple contrôle par rôle).

MVP — *Minimum Viable Product* : périmètre fonctionnel minimal mais complet, défini en amont du développement.